

WebAR Split Video (Alpha Masking) Guide

This document is a complete technical guide to replacing real-time green screen (chroma key) shaders with the highly optimized **Split Video (Side-by-Side Alpha Masking)** technique in Unity WebGL.

1. Why Use Split Video?

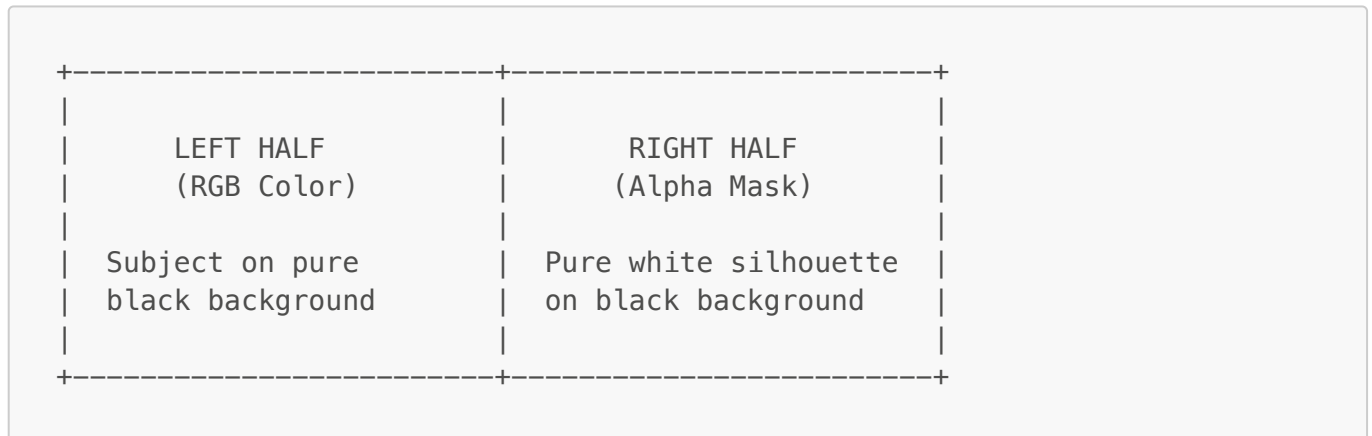
In WebAR, playing video with transparency is notoriously difficult.

- Standard **.mp4** files do not support transparency.
- Transparent **.webm** works on Android/Chrome but breaks on iOS/Safari.
- Transparent HEVC **.mp4** works on iOS/Safari but breaks on Android/Chrome.
- Real-time chroma keying (green screen shaders) inside Unity causes "green halos", jagged edges, and struggles with semi-transparent pixels like shadows, smoke, or glass.

The Solution: We create a double-wide standard **.mp4** file. One half holds the color data, and the other half holds a black-and-white map of the transparency. Because it's a standard MP4, it plays at 60fps on **every device and browser** with perfect, artifact-free edges.

2. The Video Structure

Instead of a standard **1920x1080** video, you render a **3840x1080** video.



3. How to Create the Video (After Effects Workflow)

1. **Key the Footage:** Import your raw green screen footage into After Effects and use **Keylight** to perfectly remove the green background.
2. **Setup Composition:** Change your Composition Settings to be exactly twice as wide as your footage (e.g., if footage is 1080x1080, make the comp 2160x1080).
3. **Left Side (Color):** Place your keyed footage on the left half of the composition. Ensure the background behind it is pure black.
4. **Right Side (Mask):** Duplicate the footage layer and move it to the right half.
5. **Create the Matte:** Apply the **Fill** effect to the right-side footage and set the color to pure White (**#FFFFFF**). Now you have a white silhouette. (If your subject has semi-transparent elements like

shadows, use a Luma Matte or Channel manipulation so gray pixels represent partial transparency).

6. **Export:** Render the video using Adobe Media Encoder as a standard H.264 (.mp4) file with a moderate bitrate (e.g., 2-4 Mbps for mobile WebAR).

4. The Unity Implementation

To make this work in Unity, you apply the video to a Quad or Plane, and attach a material using the custom shader below.

The Shader Code

Create a new file in your Unity project named `SplitVideoAlpha.shader` and paste this code:

```
Shader "Custom/SplitVideoAlpha"
{
    Properties
    {
        _MainTex ("Video Texture", 2D) = "white" {}
        [Toggle(INVERT_MASK)] _InvertMask("Invert Mask (Black=Visible)",
Float) = 0
    }
    SubShader
    {
        Tags { "RenderType"="Transparent" "Queue"="Transparent"
"IgnoreProjector"="True" }
        LOD 100

        // Standard Alpha Blending
        Blend SrcAlpha OneMinusSrcAlpha
        ZWrite Off
        Cull Off

        Pass
        {
            CGPROGRAM
            #pragma vertex vert
            #pragma fragment frag
            #pragma shader_feature INVERT_MASK

            #include "UnityCG.cginc"

            struct appdata
            {
                float4 vertex : POSITION;
                float2 uv : TEXCOORD0;
            };

            struct v2f
            {
                float2 uv : TEXCOORD0;
                float4 vertex : SV_POSITION;
```

```

    };

    sampler2D _MainTex;
    float4 _MainTex_ST;

    v2f vert (appdata v)
    {
        v2f o;
        o.vertex = UnityObjectToClipPos(v.vertex);
        // Map the mesh UVs to the LEFT half of the video texture
        o.uv = TRANSFORM_TEX(v.uv, _MainTex);
        o.uv.x *= 0.5;
        return o;
    }

    fixed4 frag (v2f i) : SV_Target
    {
        // Sample the RGB color from the left half
        fixed4 col = tex2D(_MainTex, i.uv);

        // Calculate the UV coordinate for the right half (add 0.5
to X)

        float2 maskUV = i.uv;
        maskUV.x += 0.5;

        // Sample the mask pixel from the right half
        fixed4 maskPixel = tex2D(_MainTex, maskUV);

        // The alpha value is determined by the red channel of the
mask

        float alpha = maskPixel.r;

        #if INVERT_MASK
            alpha = 1.0 - alpha;
        #endif

        // Apply the alpha to the color
        col.a = alpha;

        return col;
    }
    ENDCG
}
}
}

```

Unity Setup Steps

1. Import your **SplitVideoAlpha.shader** into Unity.
2. Right-click the shader -> **Create -> Material**. Call it **SplitVideoMat**.
3. Create a **Quad** or **Plane** in your AR scene. (Note: Since the shader maps only the left half, you do *not* need to double the width of the Quad. Keep it at the aspect ratio of the left half).

4. Assign **SplitVideoMat** to the Quad.
5. Set up your Unity **Video Player** component to render to a **Render Texture**.
6. Assign that Render Texture to the **Video Texture** slot of your **SplitVideoMat**.
7. Hit Play!

You will now have a perfectly transparent video playing in AR without any green screen shaders.